10-03-2026_Pandas Introduction

Country GDP analysis project

In [1]:
```
import pandas as pd
```

In [2]:
```
pd.__version__
```

Out[2]:
```
'2.3.3'
```

In [3]:
```
df = pd.read_csv(r'C:\Users\mehar\OneDrive\Desktop\Class\Daily Home work given\016.pandas introduction_10-03-2026\10th - pandas introduction\data.csv')
```

In [4]:
```
df
```

Out[4]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [5]:
```
type(df)
```

Out[5]:
```
pandas.core.frame.DataFrame
```

In [6]:
```
len(df)
```

Out[6]: 195

In [7]:
```
df.columns
```

Out[7]:
```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
```

In [8]:
```
len(df.columns)
```

Out[8]: 5

In [9]:
```
# dimentions of the table
df.shape
```

Out[9]: (195, 5)

In [10]:
```
# check the missing values
df.isnull()
```

Out[10]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
190	False	False	False	False	False
191	False	False	False	False	False
192	False	False	False	False	False
193	False	False	False	False	False
194	False	False	False	False	False

195 rows × 5 columns

In [11]:
```
df.isnull().sum() # will give the no.of null values in the column
```

Out[11]:
```
CountryName      0
CountryCode      0
BirthRate        0
InternetUsers    0
IncomeGroup      0
dtype: int64
```

In [12]:
```
df.head() # by default it print top 5 rows
```

Out[12]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income

In [13]:
```
df.head(2) # to get top 2 rows
```

Out[13]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income

In [14]:
```
df.tail() # by default it print bottom 5 rows
```

Out[14]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

In [15]:
```
df.dtypes
```

Out[15]:
```
CountryName       object
CountryCode       object
BirthRate        float64
InternetUsers    float64
IncomeGroup       object
dtype: object
```

- internally all categorical data system will consider as object

In [16]:
```
df.info() # information of the dataframe
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CountryName    195 non-null    object
 1   CountryCode    195 non-null    object
 2   BirthRate      195 non-null    float64
 3   InternetUsers  195 non-null    float64
 4   IncomeGroup    195 non-null    object
dtypes: float64(2), object(3)
memory usage: 7.7+ KB
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

11-03-2026

Country GDP analysis project_Slicing in pandas

In [17]:
```
df[:]
```

Out[17]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [18]:
```
df[:6] # top 5 records
```

Out[18]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
5	Argentina	ARG	17.716	59.9	High income

In [19]:
```
df[3:] # df[3] will through you and error
```

Out[19]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
5	Argentina	ARG	17.716	59.9	High income
6	Armenia	ARM	13.308	41.9	Lower middle income
7	Antigua and Barbuda	ATG	16.447	63.4	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

192 rows × 5 columns

In [20]:
```
df[0:50:5]
```

Out[20]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.90	High income
5	Argentina	ARG	17.716	59.90	High income
10	Azerbaijan	AZE	18.300	58.70	Upper middle income
15	Bangladesh	BGD	20.142	6.63	Lower middle income
20	Belarus	BLR	12.500	54.17	Upper middle income
25	Barbados	BRB	12.188	73.00	High income
30	Canada	CAN	10.900	85.80	High income
35	Cameroon	CMR	37.236	6.40	Lower middle income
40	Costa Rica	CRI	15.022	45.96	Upper middle income
45	Germany	DEU	8.500	84.17	High income

In [21]:
```
df[::10] # df[::-10] prints from the 194th country (backward step-slicing)
```

Out[21]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.900000	High income
10	Azerbaijan	AZE	18.300	58.700000	Upper middle income
20	Belarus	BLR	12.500	54.170000	Upper middle income
30	Canada	CAN	10.900	85.800000	High income
40	Costa Rica	CRI	15.022	45.960000	Upper middle income
50	Ecuador	ECU	21.070	40.353684	Upper middle income
60	Gabon	GAB	30.555	9.200000	Upper middle income
70	Greenland	GRL	14.500	65.800000	High income
80	India	IND	20.291	15.100000	Lower middle income
90	Kazakhstan	KAZ	22.730	54.000000	Upper middle income
100	Libya	LBY	21.425	16.500000	Upper middle income
110	Moldova	MDA	12.141	45.000000	Lower middle income
120	Mozambique	MOZ	39.705	5.400000	Low income
130	Netherlands	NLD	10.200	93.956400	High income
140	Poland	POL	9.600	62.849200	High income
150	Sudan	SDN	33.477	22.700000	Lower middle income
160	Suriname	SUR	18.455	37.400000	Upper middle income
170	Tajikistan	TJK	30.792	16.000000	Lower middle income
180	Uruguay	URY	14.374	57.690000	High income
190	Yemen, Rep.	YEM	32.947	20.000000	Lower middle income

In [22]:
```
df.columns
```

Out[22]:
```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
```

In [23]:
```
# describe is refering for discriptive stats
df.describe() # it will give only numerical attribute
```

Out[23]:

	BirthRate	InternetUsers
count	195.000000	195.000000
mean	21.469928	42.076471
std	10.605467	29.030788
min	7.900000	0.900000
25%	12.120500	14.520000
50%	19.680000	41.000000
75%	29.759500	66.225000
max	49.661000	96.546800

In [24]:
```
# split the data
df['CountryName']
```

Out[24]:
```
0                    Aruba
1              Afghanistan
2                   Angola
3                  Albania
4     United Arab Emirates
               ...
190            Yemen, Rep.
191           South Africa
192       Congo, Dem. Rep.
193                 Zambia
194               Zimbabwe
Name: CountryName, Length: 195, dtype: object
```

In [25]:
```
df[['CountryName', 'CountryCode']]
```

Out[25]:

	CountryName	CountryCode
0	Aruba	ABW
1	Afghanistan	AFG
2	Angola	AGO
3	Albania	ALB
4	United Arab Emirates	ARE
...	...	...
190	Yemen, Rep.	YEM
191	South Africa	ZAF
192	Congo, Dem. Rep.	COD
193	Zambia	ZMB
194	Zimbabwe	ZWE

195 rows × 2 columns

In [26]:
```
df[['CountryName', 'CountryCode', 'IncomeGroup']]
```

Out[26]:

	CountryName	CountryCode	IncomeGroup
0	Aruba	ABW	High income
1	Afghanistan	AFG	Low income
2	Angola	AGO	Upper middle income
3	Albania	ALB	Upper middle income
4	United Arab Emirates	ARE	High income
...	...	...	...
190	Yemen, Rep.	YEM	Lower middle income
191	South Africa	ZAF	Upper middle income
192	Congo, Dem. Rep.	COD	Low income
193	Zambia	ZMB	Lower middle income
194	Zimbabwe	ZWE	Low income

195 rows × 3 columns

In [27]:
```
# creating a variable name
df_cat = df[['CountryName', 'CountryCode', 'IncomeGroup']]
df_cat
```

Out[27]:

	CountryName	CountryCode	IncomeGroup
0	Aruba	ABW	High income
1	Afghanistan	AFG	Low income
2	Angola	AGO	Upper middle income
3	Albania	ALB	Upper middle income
4	United Arab Emirates	ARE	High income
...	...	...	...
190	Yemen, Rep.	YEM	Lower middle income
191	South Africa	ZAF	Upper middle income
192	Congo, Dem. Rep.	COD	Low income
193	Zambia	ZMB	Lower middle income
194	Zimbabwe	ZWE	Low income

195 rows × 3 columns

In [28]:
```
df_cat = df[['CountryName', 'CountryCode', 'IncomeGroup']]
df_cat.head(2)
```

Out[28]:

	CountryName	CountryCode	IncomeGroup
0	Aruba	ABW	High income
1	Afghanistan	AFG	Low income

In [29]:
```
df_num = df[['BirthRate', 'InternetUsers']]
df_num
```

Out[29]:

	BirthRate	InternetUsers
0	10.244	78.9
1	35.253	5.9
2	45.985	19.1
3	12.877	57.2
4	11.044	88.0
...	...	...
190	32.947	20.0
191	20.850	46.5
192	42.394	2.2
193	40.471	15.4
194	35.715	18.5

195 rows × 2 columns

In [30]:
```
# 3 variables were created untill now
print(df.shape)
print(df_num.shape)
print(df_cat.shape)
```

```
(195, 5)
(195, 2)
(195, 3)
```

In [31]:
```
print(df.columns)
print(df_num.columns)
print(df_cat.columns)
```

```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
Index(['BirthRate', 'InternetUsers'], dtype='object')
Index(['CountryName', 'CountryCode', 'IncomeGroup'], dtype='object')
```

In [32]:
```
df_cat.describe()
```

Out[32]:

	CountryName	CountryCode	IncomeGroup
count	195	195	195
unique	195	195	4
top	Aruba	ABW	High income
freq	1	1	67

- here min & max are missing
- because the min & max were applied to numerical values not for categorical data

In [33]:
```
df_cat.describe().T # transpose
```

Out[33]:

	count	unique	top	freq
CountryName	195	195	Aruba	1
CountryCode	195	195	ABW	1
IncomeGroup	195	4	High income	67

In [34]:
```
# transpose is easy to read data with out scrolling up and down
df_num.describe().T
```

Out[34]:

	count	mean	std	min	25%	50%	75%	max
BirthRate	195.0	21.469928	10.605467	7.9	12.1205	19.68	29.7595	49.6610
InternetUsers	195.0	42.076471	29.030788	0.9	14.5200	41.00	66.2250	96.5468

In [35]:
```
df.columns
```

Out[35]:
```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
```

In [36]:
```
df.head(2)
```

Out[36]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income

In [37]:
```
# changing column names
df.columns = ['a', 'b', 'c', 'd','e']
```

In [38]:
```
df.head(2)
```

Out[38]:

	a	b	c	d	e
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income

- while changing columns names the length should match otherwise it will through value error
- df.colunms = ['a', 'b', 'c', 'd','e','f'] --> it gives an value error

In [39]:
```
df.columns = ['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers', 'IncomeGroup']
```

In [40]:
```
df.head(1)
```

Out[40]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income

- We can change the specific column name also

In [41]:
```
# apply conditions
df['InternetUsers'] < 2
```

Out[41]:
```
0      False
1      False
2      False
3      False
4      False
       ...
190    False
191    False
192    False
193    False
194    False
Name: InternetUsers, Length: 195, dtype: bool
```

In [42]:
```
df[df['InternetUsers'] <2] # view the full record which are <2
```

Out[42]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
11	Burundi	BDI	44.151	1.3	Low income
52	Eritrea	ERI	34.800	0.9	Low income
55	Ethiopia	ETH	32.925	1.9	Low income
64	Guinea	GIN	37.337	1.6	Low income
117	Myanmar	MMR	18.119	1.6	Lower middle income
127	Niger	NER	49.661	1.7	Low income
154	Sierra Leone	SLE	36.729	1.7	Low income
156	Somalia	SOM	43.891	1.5	Low income
172	Timor-Leste	TLS	35.755	1.1	Lower middle income

In [43]:
```
len(df[df['InternetUsers'] <2])
```

Out[43]: 9

In [44]:
```
df
```

Out[44]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [45]:
```
df['BirthRate'] > 40
```

Out[45]:
```
0      False
1      False
2       True
3      False
4      False
       ...
190    False
191    False
192     True
193     True
194    False
Name: BirthRate, Length: 195, dtype: bool
```

In [46]:
```
df[df['BirthRate'] > 40]
```

Out[46]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
2	Angola	AGO	45.985	19.1	Upper middle income
11	Burundi	BDI	44.151	1.3	Low income
14	Burkina Faso	BFA	40.551	9.1	Low income
65	Gambia, The	GMB	42.525	14.0	Low income
115	Mali	MLI	44.138	3.5	Low income
127	Niger	NER	49.661	1.7	Low income
128	Nigeria	NGA	40.045	38.0	Lower middle income
156	Somalia	SOM	43.891	1.5	Low income
167	Chad	TCD	45.745	2.3	Low income
178	Uganda	UGA	43.474	16.2	Low income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income

In [47]:
```
len(df[df['BirthRate'] > 40])
```

Out[47]:
```
12
```

In [48]:
```
# df[df['BirthRate'] > 40] & df[df['InternetUsers'] < 2] --> this doesn't work
df[(df.BirthRate > 40) & (df.InternetUsers < 2)]
```

Out[48]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
11	Burundi	BDI	44.151	1.3	Low income
127	Niger	NER	49.661	1.7	Low income
156	Somalia	SOM	43.891	1.5	Low income

In [49]:
```
df.head()
```

Out[49]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income

In [50]:
```
df['IncomeGroup'].unique() # displays Unique values in IncomeGroup
```

Out[50]:
```
array(['High income', 'Low income', 'Upper middle income',
       'Lower middle income'], dtype=object)
```

In [51]:
```
df['IncomeGroup'].nunique() # displays the no.of Unique values in IncomeGroup
```

Out[51]: 4

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

12-03-2026

Visualization

In [52]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

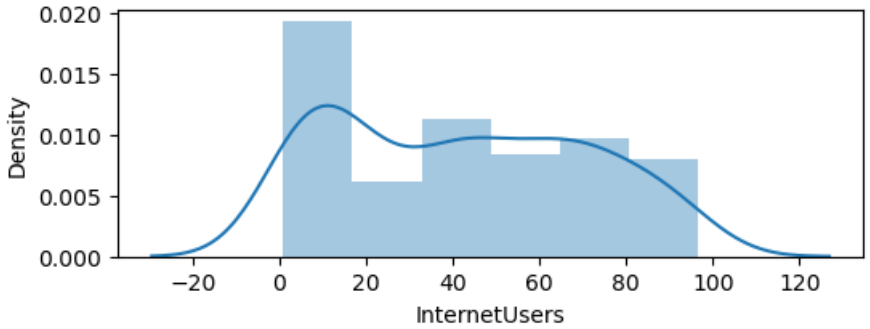
In [53]:
```
%matplotlib inline
plt.rcParams['figure.figsize'] = 6,2
```

In [54]:
```
import warnings
warnings.filterwarnings('ignore')
```

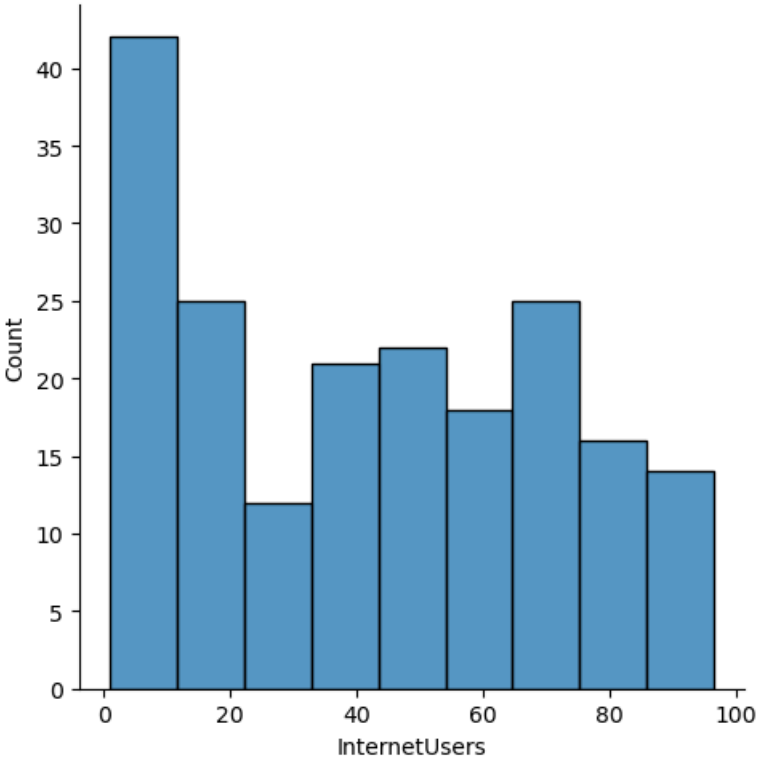
In [55]:
```
df['InternetUsers']
```

Out[55]:
```
0      78.9
1       5.9
2      19.1
3      57.2
4      88.0
       ...
190    20.0
191    46.5
192     2.2
193    15.4
194    18.5
Name: InternetUsers, Length: 195, dtype: float64
```

In [56]:
```
vis1 = sns.distplot(df["InternetUsers"])
```

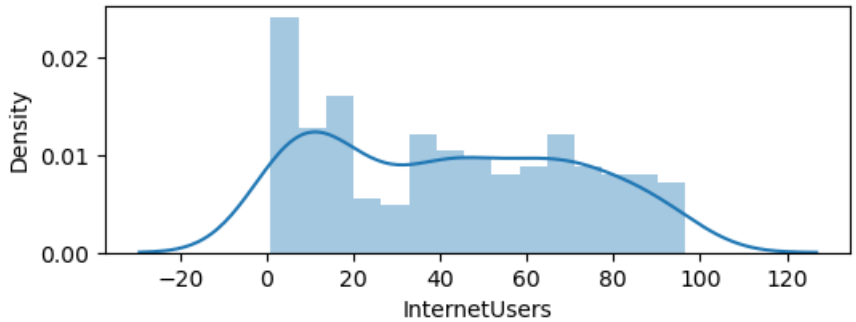


In [57]:
```
# if given displot instead of distplot the line graph was missing
vis1 = sns.displot(df["InternetUsers"])
```

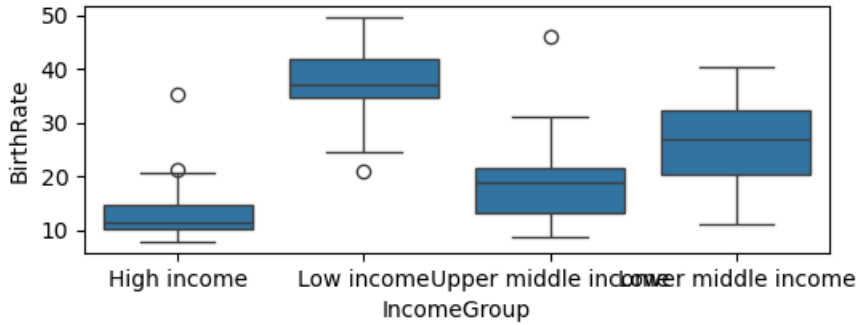


- If plt was given instead of sns it will through an error (attribute error)

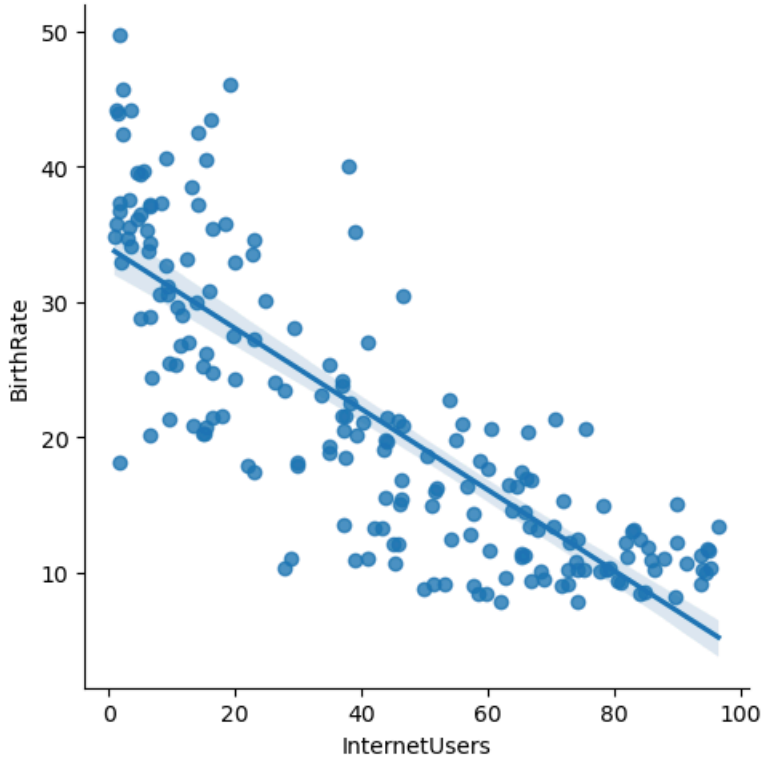
In [58]:
```
# we get 15 bins and the analysis is much better compared to previous graph
vis1 = sns.distplot(df["InternetUsers"], bins = 15)
```



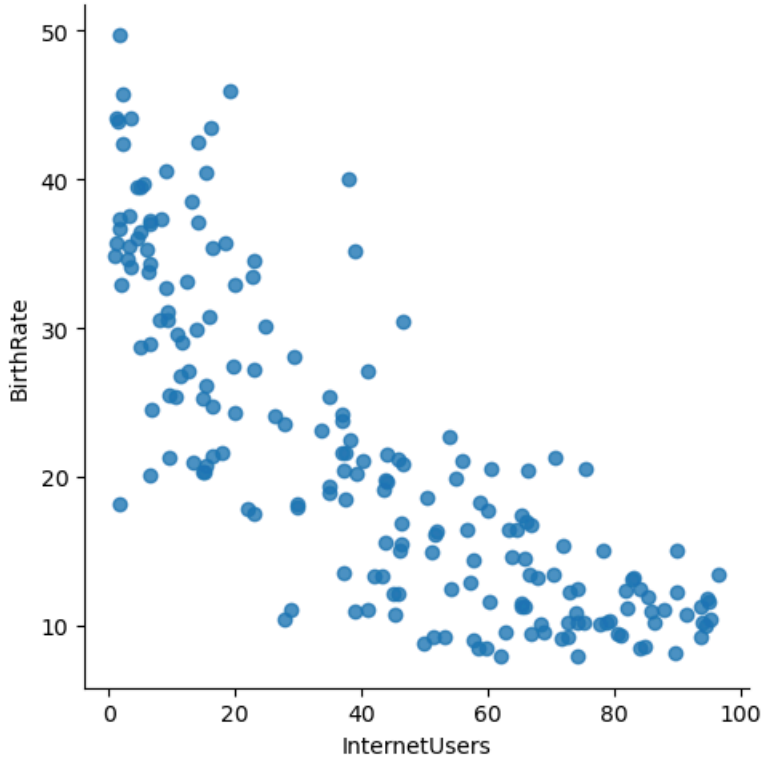
In [59]:
```
# BOX PLOTS:
vis4 = sns.boxplot(data = df, x="IncomeGroup", y="BirthRate") # BI-VARIATE ANALYSIS
```



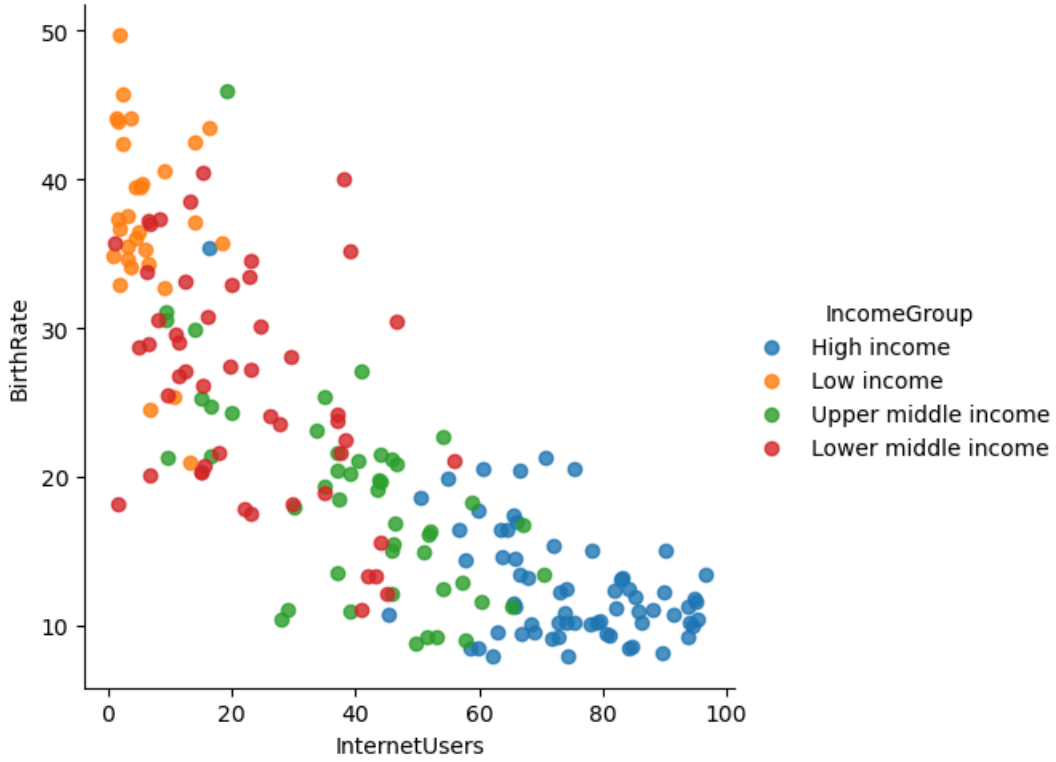
In [60]:
```
# refer to seaborn gallary
# visualizing with seaborn
vis5 = sns.lmplot(data = df, x = 'InternetUsers', y = 'BirthRate')
```



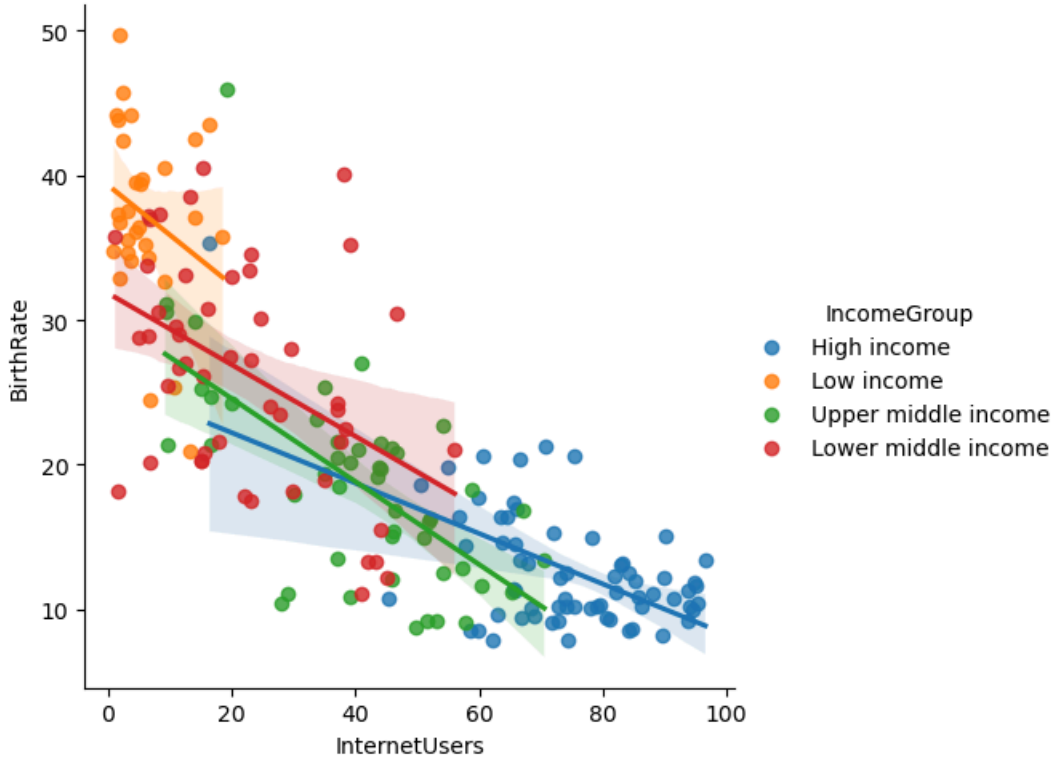
In [61]:
```
# when we use fit_reg = False --> we don't get line in graph
# to get line write True instead of False i.e., fit_reg = True
vis5 = sns.lmplot(data = df, x = 'InternetUsers', y = 'BirthRate', fit_reg = False) # lm - linear model
```



In [62]:
```
# we add color for better understand if all are in one color we can't identify pattern
# use hue it add colors to graph
vis8 = sns.lmplot(data = df, x = 'InternetUsers', y = 'BirthRate',
                  fit_reg = False, hue = 'IncomeGroup') # hue = parameter for color
```



In [63]:
```
vis5 = sns.lmplot(data = df, x = 'InternetUsers', y = 'BirthRate',
                  fit_reg = True, hue = 'IncomeGroup')
```



In this section we learned

- Importing data into python
- Dataframe via panda
- Exploring datasets: head(), tail(), info(), describe()
- Renaming columns
- Subsetting dataframes
- Basic operations with dataframe
- Filtering data frames
- Seaborn introduction -- .distplot | .boxplot | .lmplot(fit_reg) | outlier | hue parameter
- Univariate, Bivariate, Multivariate analysis